

MP2 Memory management: Kernel Memory Allocation (slab)

基本資訊

- 滿分: 140% (基本部分: 100%, 加分部分 40%)
- Release Date: 2025/03/18
- Due Date: 2025/04/01
- TA email: ntuos@googlegroups.com
- TA hours: Wed. 13-14 p.m., Fri. 11 a.m. -12 p.m., at B04

MP2 Memory management: Kernel Memory Allocation (slab)

基本資訊

Overview

環境設定與準備

評分標準與繳交方式

評分標準

基本要求

加分項目

提交與評分方式

問題背景：核心記憶體管理的挑戰

Slab 配置器的初步設計與 API

Slab 配置器的初步設計

API 介面

1. `kmem_cache_create`
2. `kmem_cache_alloc`
3. `kmem_cache_free`

Slab 配置器的實作的考量點

`freelist` 資料結構

鏈結串列的應用

如何為連續記憶體建立鏈結串列？

`freelist` 的元素數量

`kmem_cache` 資料結構

核心物件、`struct slab` 與 `struct kmem_cache` 的關係

核心物件

`struct slab`

`struct kmem_cache`

`freelist` 與 `slab` 的關係

`kmem_cache` 的同步控制與競爭議題

`freelist` 的安全性強化 (可選的加分項目)

(1) `freelist` 指標混淆 (Pointer Obfuscation)

(2) `freelist` 隨機化 (Freelist Randomization)

`kmem_cache` 的內部碎裂問題 (加分項目)

實作前需詳閱的資訊

- 檔案修改規則
- 簡易核心除錯器
- 動態切換除錯模式
- 調整預設除錯模式

實作要求

- `struct slab` 設計
- `struct kmem_cache` 的設計
- Slab 提供的功能
- `kmem_cache_create`: 創建 `kmem_cache`
- `kmem_cache_alloc`: 配置物件
- `kmem_cache_free`: 釋放物件
- 將 slab 配置器應用於 `struct file` 的管理
- `print_kmem_cache` 列印 `struct kmem_cache` 的資訊
- 實作 system call `sys_printfslab`

參考資料

Overview

在 MP2 作業中，我們將探討 核心小型物件的記憶體配置與釋放機制，並請學生在 xv6 作業系統上 設計並實作 SLAB 分配器。

本次作業的重點包括：

- SLAB 分配器的資料結構設計
- 小型物件分配的時間與空間效率優化
- 與 SLAB 相關的多種優化技術

透過本作業，學生將能夠體驗 系統記憶體管理的設計與實作過程，並學習如何提升核心記憶體配置的效率。

環境設定與準備

請確認以下步驟，以確保開發環境設置完成：

1. 確保已安裝 [Git](#)
2. 確保擁有 [GitHub 帳號](#)，若無，請先註冊
3. 透過 MP2 專屬 [GitHub Classroom 連結](#)，點擊 **Accept this assignment**，系統將為您建立專屬的作業倉庫 `mp2-<USERNAME>`
4. 存取您的 MP2 倉庫 `https://github.com/ntuos2025/mp2-<USERNAME>`
5. 在本地端克隆倉庫：

```
git clone https://github.com/ntuos2025/mp2-<USERNAME>
```

6. 在倉庫內的 `student_id.txt` 檔案中填入您的學號，例如：

```
b12345678
```

7. 運行 `mp2.sh`，此腳本將準備 `ntuos/mp2` 容器：

```
./mp2.sh start
```

8. (選擇性) 容器內 VS Code 開發環境設置：

- 安裝 [Docker](#) 及 [Dev Containers](#) 擴充套件
- 開啟 VS Code，進入 **Docker** 側邊欄，找到 `ntuos/mp2`
- 右鍵點擊 **Attach Visual Studio Code**
- 選擇 `ntuos/mp2`，此時 VS Code 會開啟新的開發環境，可直接於容器內進行開發

9. 測試環境是否正常運行：

```
./mp2.sh test
```

評分標準與繳交方式

評分標準

本作業總分 **140%**，其中包含 基本要求與加分項目。建議學生 先完成基本實作，再挑戰額外的加分項目。

基本要求

- 除錯工具 (10%)
 - `sys_printfslab` 系統呼叫 (5%)
 - `print_kmem_cache` (5%)
- [SLAB 設計](#) (5%)
- 功能測試 (Public Tests) (45%)
 - `kmem_cache_create` (5%)
 - `kmem_cache_alloc` (20%)
 - `kmem_cache_alloc` + `kmem_cache_free` (20%)
- 隱藏測試 (Private Tests) (40%)

加分項目

加分項目與基本要求不衝突，且加分項目不互相影響，可以選擇實現多個項目。

- 安全性增強 (最高 +16%)
 - `freelist` 指標混淆 (+8%)
 - `freelist` 隨機化 (+8%)
- SLAB 優化 (最高 +24%)
 - `struct slab` 記憶體優化 (+6%)

- `kmem_cache` 內部碎裂優化 (+8%)
- 以 `kernel/list.h` 管理 SLAB (+10%)

提交與評分方式

所有程式碼將透過 git 繳交至 **GitHub Classroom**，請確保您的最終提交符合規範。

在截止時間之前可以繳交無數次，作業將透過我們準備好的 Github Action 自動評分，其會檢查提交紀錄與測試結果，並記錄最終得分。

最終得分將包含功能測試與隱藏測試，可至同學 mp2 的 Repository 中點選 Github Action 查看運行結果。

問題背景：核心記憶體管理的挑戰

想像今天 kernel 要分配 100 個大小為 40B 個系統物件，比如 xv6 中的 `struct file`。若每個物件都要以一個頁面儲存，不僅需要配置頁面時的開銷，還會遇到非常嚴重的內部碎裂問題，特別是面對小型系統物件的配置。

若能利用這些系統物件大小相同的特性，核心預先分配一個乃至數個連續的頁面，並將其內部空間以 40B 為單位進行切割，便能盡可能地利用整個頁面的空間，減少內部碎裂，由於可能短時間內對相同頁面重複訪問，且大多數的配置可以重複利用舊的頁面而非配置新頁面，也能加速配置所需的時間。

Slab Allocation

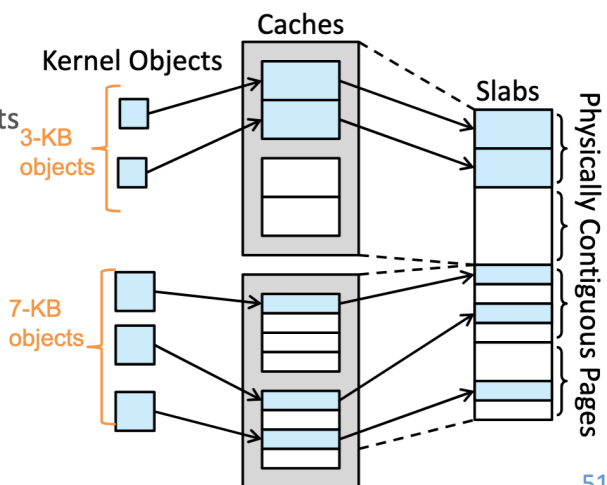
❑ A **slab** is made up of one or more physically contiguous pages

❑ A **cache** consists of one or more slabs

- There is a single cache for each unique kernel data structure
 - Examples: process descriptors (process control blocks), file objects, ...
 - E.g., 2 kernel objects 3 KB in size and 3 objects 7 KB in size, each stored in a separate cache in the example below.

❑ Each cache is populated with **objects**

- Instantiations of the kernel data structure that the cache represents
 - e.g., the cache representing PCBs stores instances of PCB objects.
- The number of objects in the cache depends on the size of the associated slab.
 - e.g., a 12-KB slab (made up of three contiguous 4-KB pages) could store six 2-KB objects.



51

回顧課程投影片。

Slab 便是這樣一個系統，源自 SunOS 原始碼，為過去 Linux kernel 實現小型系統物件記憶體配置的機制。本作業將引導學生設計並實作一個新的 **Slab** 記憶體配置系統，以提升小型物件的記憶體管理效能。

題外話，MP2 的助教們所屬的實驗室簡稱為 NEWSLAB，可以斷句為 NewSlab，這便是我們 MP2 的目標！嗯...希望不要太冷。

Slab 配置器的初步設計與 API

Slab 配置器的初步設計

在 Slab 配置器的設計中，我們的目標是：

1. 為具有相同大小的核心物件分配頁面。
2. 將頁面切分為固定大小的物件，以提高記憶體使用效率。

為簡化設計，假設核心一次僅能分配單一頁面，而無法一次配置多個連續頁面。基於此假設，我們定義 `struct slab`，其大小等同於單個頁面，並包含物件的類別名稱 `name`、每個物件的大小 `object_size`，以及管理可用物件的 `freelist`。

```
struct slab {
    char name[MP2_CACHE_MAX_NAME];
    uint object_size;
    void **freelist;
    ...
};
```

在 C 語言中，`void *` 表示指向任意類型的指標，然而由於其型別不確定，無法直接解引用，因此需要將其轉型為特定類型的指標才能進行操作。`(void *) *freelist` 代表一個 `void *` 陣列，或是一個指向 `void *` 的指標。關於 `freelist` 的詳細結構，將在[後續章節](#)進一步說明。

由於對於相同類型的系統物件而言，`name` 和 `object_size` 皆為相同屬性，因此可在 `struct slab` 之上設計一個額外的結構體 `struct kmem_cache`，用來統一管理相同類型物件的共用資訊，藉此減少 `struct slab` 的記憶體開銷。

```
struct slab {
    void **freelist;
    ... // 其他成員可依需求擴充
};

struct kmem_cache {
    char name[MP2_CACHE_MAX_NAME];
    uint object_size;
    struct slab *slab; // 記錄當前使用的 slab
    ... // 其他成員可依需求擴充
};
```

我們期望這些結構體能夠提供以下四項核心功能：

```
// 初始化一個 slab 配置器，管理名稱為 name、大小為 object_size 的系統物件
struct kmem_cache *kmem_cache_create(char *name, uint object_size);

// 分配一個系統物件並回傳
void *kmem_cache_alloc(struct kmem_cache *cache);

// 釋放一個先前分配的系統物件 obj
void kmem_cache_free(struct kmem_cache *cache, void *obj);

// 銷毀 kmem_cache (不列入評分範圍)
void kmem_cache_destroy(struct kmem_cache *cache);
```

使用這些 API，其他核心開發者可透過以下方式進行記憶體管理：

```
// 初始化 struct file 的 slab 配置器
struct kmem_cache *file_cache = kmem_cache_create("file", sizeof(struct file));

// 分配一個 struct file 物件
struct file *file_allocated = (struct file *) kmem_cache_alloc(file_cache);

// 釋放一個 struct file 物件
kmem_cache_free(file_cache, file2release);

// 當不再需要 file_cache 時，將其銷毀
kmem_cache_destroy(file_cache);
```

這是 MP2 作業中 Slab 配置器的介面設計，也是本次實作的核心目標。在接下來的章節中，我們將深入探討其具體實作細節。

API 介面

本次作業應提供以下 slab API，以供 xv6 核心使用：

1. kmem_cache_create

```
struct kmem_cache *kmem_cache_create(const char *name, size_t size);
```

- 功能: 初始化一個 kmem_cache，用於管理特定大小的物件。
- 參數:
 - name：快取名稱 (供除錯與管理使用)。
 - size：物件大小 (單一物件的記憶體需求)。
- 回傳值: 指向新建立的 kmem_cache 的指標。

2. kmem_cache_alloc

```
void *kmem_cache_alloc(struct kmem_cache *cache);
```

- 功能: 從 `kmem_cache` 取得可用物件。
- 參數:
 - `cache` : 指向 `kmem_cache` 結構的指標。
- 回傳值: 成功時回傳指向已分配物件的指標，失敗時回傳 `NULL`。

3. `kmem_cache_free`

```
void kmem_cache_free(struct kmem_cache *cache, void *obj);
```

- 功能: 將物件歸還至 `kmem_cache`，使其可供後續分配。
- 參數:
 - `cache` : 指向 `kmem_cache` 結構的指標。
 - `obj` : 指向待釋放物件的指標。

Slab 配置器的實作的考量點

freelist 資料結構

在閱讀前述內容後，您或許已經迫不及待地想在 xv6 中實作 SLAB 記憶體分配機制。然而，在實作過程中，如何設計 **freelist** 資料結構 將是一個關鍵挑戰。

首先，**freelist** 本質上是一塊連續的記憶體區域，亦即一個陣列。對於任何陣列來說，獲取可用元素或釋放元素的時間複雜度通常為 $O(n)$ ，其中 n 為 **freelist** 中可放置的的最大總物件數。然而，在追求高效能的 Linux 核心中，如此高的複雜度顯然不可接受。因此，我們需要採用 更適合的資料結構 來優化分配與釋放的時間。

鏈結串列的應用

學習過資料結構的讀者應該熟悉，鏈結串列 允許在 $O(1)$ 的時間內進行元素的移除和插入，可以分別用來實作物件的分配與釋放，這正是 **freelist** 的命名由來。我們可以將 **freelist** 視為一個尚未分配物件的鏈結串列，並透過以下方式進行操作：

- 分配物件：從 **freelist** 取出第一個可用物件，並更新 **freelist** 指標。
- 釋放物件：將釋放的物件插入 **freelist**。

如此一來，物件分配與釋放的時間複雜度均為 $O(1)$ ，大幅提升效能。

如何為連續記憶體建立鏈結串列？

一種直觀的做法是 額外建立一個鏈結串列來記錄陣列中的可用記憶體地址。假設原本的物件陣列為 `objects`，而 **freelist** 用於存放可用物件的地址，結構如下：

```
struct slab {
    void **objects;
    void **freelist; // 鏈結串列，初始時記錄所有可用物件的地址
};
```

然而，這種設計存在兩個問題：

1. 需要額外的記憶體來存放 freelist 的鏈結節點。
2. freelist 本身的記憶體配置與管理問題依然尚未解決。

對這類問題，Linux 核心開發者常利用 C 語言一塊記憶體各自表述的技巧，直接將尚未分配的記憶體用於存儲鏈結串列的指標，最大限度地利用已經分配的記憶體。具體而言，每個未使用的物件可以直接存放指向下一個可用物件的指標，從而形成鏈結串列。概念上與前述設計相同，但能一次解決前述設計所遇到的問題。

假設系統物件的大小皆不小於一個指標的大小，則我們可以安全地將尚未被分配的物件視為鏈結串列節點。以下是 freelist 的運作示例：

```
struct slab {
    void **freelist;
    ...
};

struct slab *s = ...;
// 取得第一個可用物件
void *free_obj = s->freelist;
// `free_obj` 的第一個指標大小的空間存放的是下一個可用物件的地址
void *next_free_obj = *(void **)free_obj;
// 重新解釋記憶體為具體的系統物件（例如 struct file）
struct file *f = (struct file *) free_obj;
struct file *f_next = (struct file *) next_free_obj;
```

太多 void * 看起來很眼花撩亂嗎？xv6 中 `kernel/kalloc.c` 提供一個可讀性高的實作方式。

```
// in kalloc.c
// struct run 是將一塊記憶體解釋為一個有 next 成員的結構體
struct run {
    struct run *next;
};

struct {
    struct spinlock lock;
    struct run *freelist;
} kmem;
```

如此一來前面的用例可以等價的改寫為


```

struct slab {
    struct run *freelist;
    ...
};

struct slab *s = ...;
// 取得第一個可用物件的指標
struct run *r = s->freelist;
// 由於第一個可用物件中 `next` 成員藏有下一個可用物件的地址
struct run *r_next = r->next;
// 重新解釋記憶體為特定的系統物件 (比如 struct file)
struct file *f = (struct file *) r;
struct file *f_after_f = (struct file *) r_next;

```

同學們也可以在一個核心物件的空間內放兩個指標，實現雙向鏈結串列。請根據 [Slab 配置器的實作要求](#)，為 freelist 設計合適資料結構。

freelist 的元素數量

考慮一個 slab 佔用一個頁面，freelist 中應該有

$$\frac{\text{Page size} - \text{Metadata size in a slab}}{\text{Size of each object}}$$

個元素。

kmem_cache 資料結構

kmem_cache 是 Slab 配置器中負責管理同類型物件記憶體分配的核心結構，其雛形如下：

```

struct slab {
    <ptr> freelist;
    ... // 其他欄位可依需求擴充
};

struct kmem_cache {
    char name[MP2_CACHE_MAX_NAME];
    uint object_size;

    struct slab *slab; // 指向當前使用的 struct slab

    ... // 其他欄位可依需求擴充
};

```

在系統設計中，需注意以下幾點：

- struct kmem_cache (以及 struct slab) 的大小為一個頁面，為固定值，無法動態調整。

- 为了提高記憶體分配效率，`struct kmem_cache` 應指向尚有可用空間的 Slab 清單，常見 Slab 清單分類如下：
 - 已滿 (**full**)：所有物件皆已分配。
 - 部分使用 (**partial**)：仍有可用物件。
 - 空閒 (**free**)：尚未使用的 Slab。
- 當所有現有 Slab 皆已滿時，應配置新的頁面以產生額外的 Slab。因此，儘管每個 `kmem_cache` 僅對應單一物件類型，其所管理的 Slab 數量可能會隨記憶體需求動態變化。

由於 `kmem_cache` 可能需動態管理大量 Slab，因此將 `kmem_cache::slab` 設計為陣列並不實際，改採 鏈結串列 會更符合需求。此設計可改進為：

```
struct slab {
    <ptr> freelist;

    {    // Slab 清單鏈結指標
        <ptr> <next>;
        <ptr> <prev>; // 可選，視情境需求決定是否使用
    }
    ... // 其他欄位可依需求擴充
};

struct kmem_cache {
    char name[MP2_CACHE_MAX_NAME];
    uint object_size;

    <ptr> full;    // 指向已滿的 Slab 清單
    <ptr> partial; // 指向部分使用的 Slab 清單
    <ptr> free;    // 指向空閒的 Slab 清單

    ... // 其他欄位可依需求擴充
};
```

其中 `<ptr>` 可為 `struct slab *`、`void *`，或 `struct list_head` 等型別，後者為 Linux 風格的雙向鏈結串列管理方式，提供於 `kernel/list.h`。如需瞭解 `kernel/list.h` 的使用方式，請參考該檔案內的文檔註釋。採用該函式庫提供的 `struct list_head` 實作 slab 清單的同學將獲得加分機會。

核心物件、*struct slab* 與 *struct kmem_cache* 的關係

在 MP2 的 SLAB 配置器中，有三個關鍵角色，其關係應予以明確釐清。

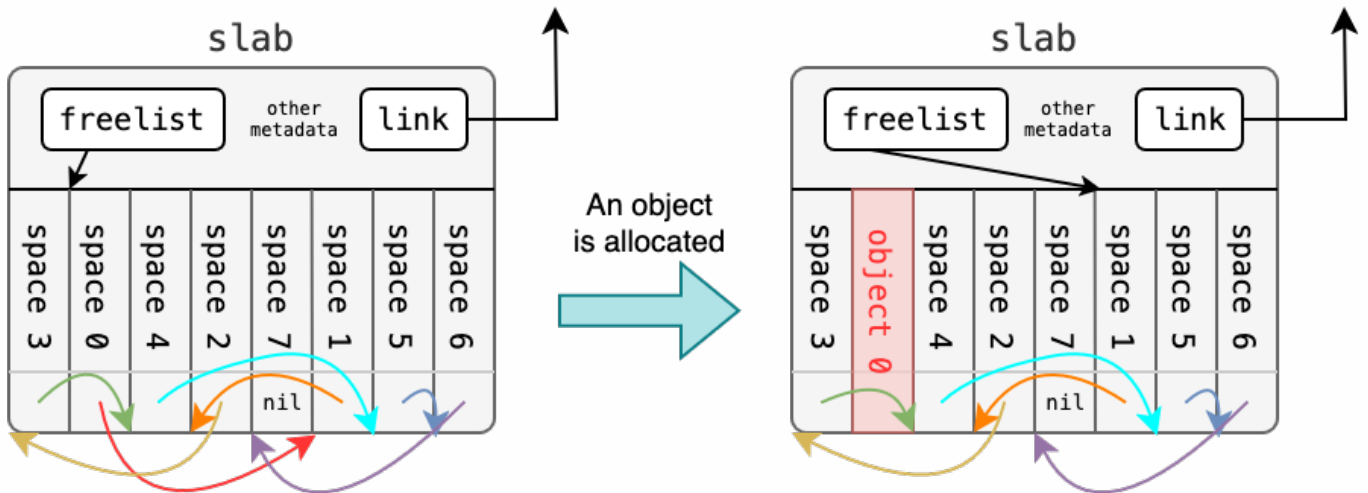
核心物件

核心所管理的資料結構，例如 `struct file`。每個核心物件的大小皆為固定長度。

struct slab

- 佔用單個頁面的記憶體區塊，其中部分空間存放 Slab 的 元數據 (**metadata**)，其餘空間則被切割為大小相同的記憶體單元，形成 **freelist** (可用物件清單)。freelist 內的每個單元可存放一個核心物件。
- 不同類型的核心物件會存放於對應類型的 Slab 中，以確保管理的獨立性。

- 根據已分配物件的數量，`struct slab` 可分為三種狀態：
 - 滿 (full)：所有可用物件均已分配。
 - 部分使用 (partial)：部分物件已被分配，仍有可用空間。
 - 閒置 (free)：目前無任何物件被分配，可完全釋放。
- 定義「可用的 Slab」為處於 **partial** 或 **free** 狀態的 **Slab**，這些 Slab 仍可提供物件分配。
- `struct slab` 依需求動態分配，並透過鏈結串列進行管理，確保可靈活擴展。

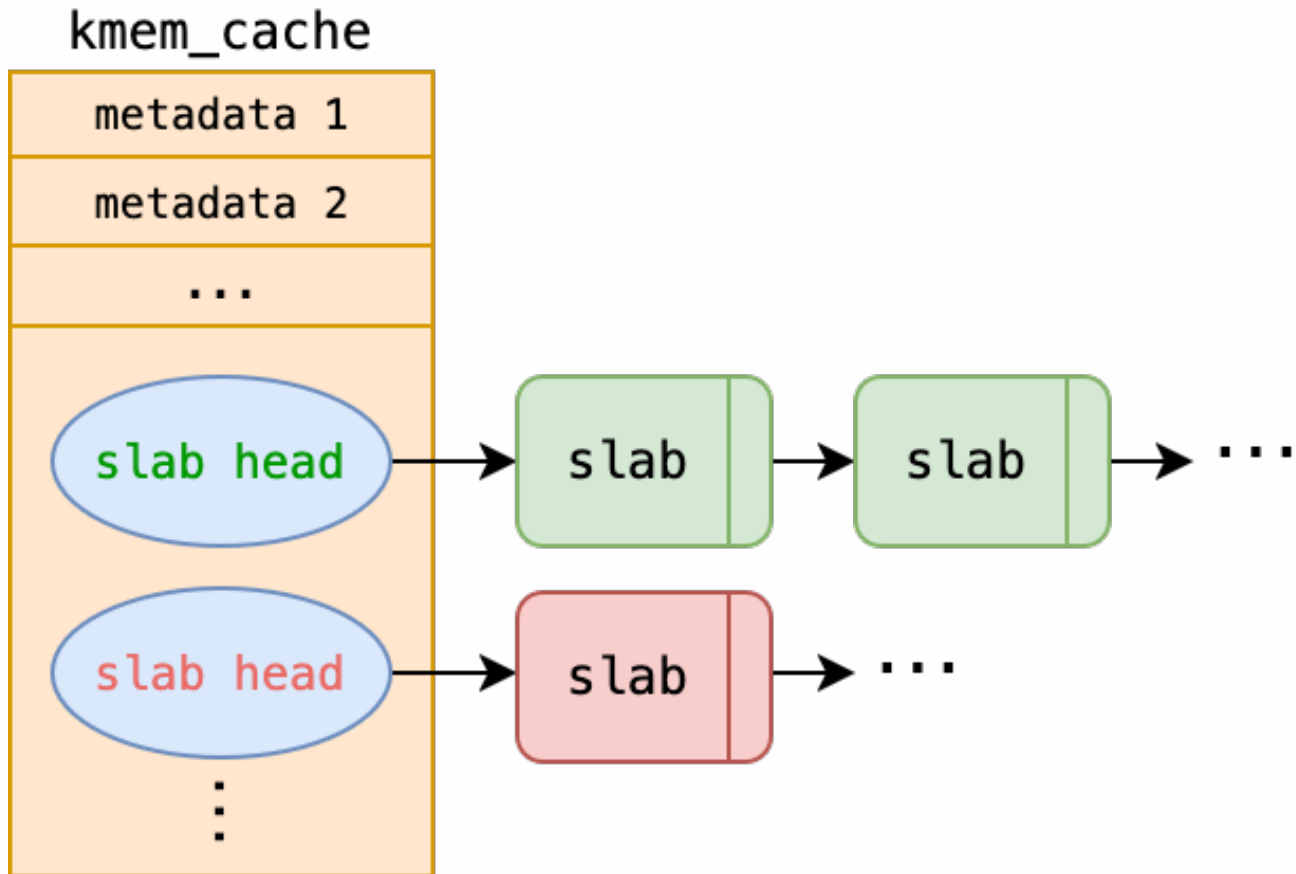


上圖便是一個閒置的 slab 配置一個物件前後的狀態變化示意圖。配置完後變成一個部分使用的 slab。

此外，在 `kmem_cache_free` 的函式簽名中，僅提供 `struct kmem_cache *cache` 及核心物件指標 `void *obj` 的情況下，如何確定 `obj` 所屬的 `struct slab` 是一項重要的設計課題。請思考有哪些設計方案或技術可用於實現 `obj` 到 slab 的映射？

`struct kmem_cache`

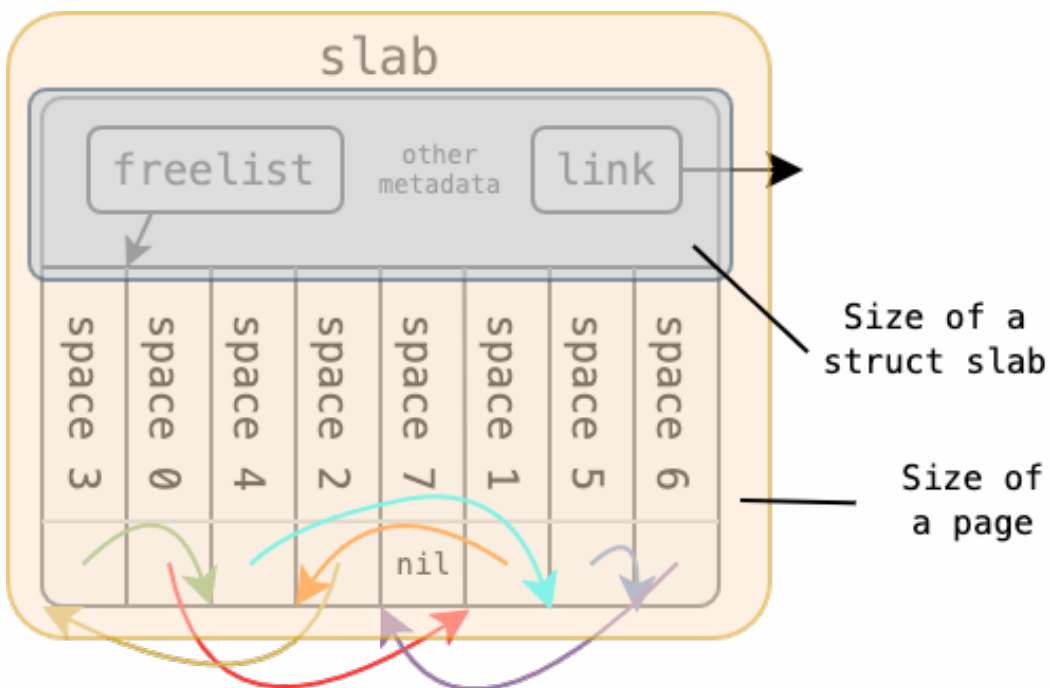
- 負責管理所有屬於同一類型核心物件的 **Slab**，確保記憶體分配與釋放的高效運行。
- 例如，系統可能為 `struct file` 和 `struct proc` 各自建立一個 `kmem_cache`，專門管理其對應的 Slab。(本次作業不涉及 `struct proc` 相關程式碼)



由上圖可見 `kmem_cache` 可能管理多個元數據，並管理一至多個 `slab` 鏈結序列。

freelist 與 slab 的關係

有效存取 `freelist` 管理的記憶體空間，需對結構體與頁面記憶體的佈局有清晰的理解。下圖展示了 Slab 的記憶體佈局示意圖：



請思考如何存取圖中各個物件對應的記憶體區域 (`space 0 ~ 7`)。

提示：[指標運算 \(Pointer Arithmetic\)](#)。

kmem_cache 的同步控制與競爭議題

在多核心與多執行緒環境中，*kmem_cache* 的操作涉及對 Slab 清單的修改與管理，因此可能會產生競爭條件 (race condition)。當多個 CPU 同時存取 *kmem_cache*，特別是在執行物件分配 (*kmem_cache_alloc()*) 或釋放 (*kmem_cache_free()*) 操作時，若未採取適當的同步機制，可能導致資料不一致或記憶體損壞。

為了確保 *kmem_cache* 的執行緒安全性，應使用適當的同步機制來保護關鍵區域。常見的解決方案包括：

- **自旋鎖 (spinlock)**：適用於短時間內的鎖定操作，避免進程切換開銷。
- **互斥鎖 (mutex)**：若操作時間較長，可使用互斥鎖來降低 busy waiting 的影響。
- **Per-CPU Cache**：透過每個 CPU 獨立的 *kmem_cache_cpu* 來減少跨 CPU 鎖競爭，僅在必要時進行全局同步。事實上這便是現今 Linux 核心採納之 slab 配置器的繼承者：slub 配置器的關鍵優化思路。

本作業只需要學生們以自旋鎖確保 *kmem_cache* 的執行緒安全性。以下範例展示如何透過 xv6 提供的自旋鎖確保 *kmem_cache* 在多執行緒環境下的正確性：

```
struct kmem_cache {
    ...
    struct spinlock lock; // 用於同步管理 kmem_cache
};

void some_func() {
    struct kmem_cache *cache = kmem_cache_create(...);

    // 進入臨界區，防止競爭條件
    acquire(&cache->lock);

    // 可能產生競爭的關鍵操作
    cache->partial = NULL;

    // 釋放鎖，結束臨界區
    release(&cache->lock);
}
```

在實作 slab 的功能以及應用 slab 於 `file.c` 時請務必注意執行緒安全議題。

freelist 的安全性強化 (可選的加分項目)

目前，*freelist* 內的鏈結串列節點通常按照記憶體地址順序排列，使得分配與釋放具有可預測性，可能會被攻擊者利用。因此，Linux 核心引入以下兩種技術來提高 *freelist* 的安全性：

(1) *freelist* 指標混淆 (Pointer Obfuscation)

透過對 *freelist* 指標進行 XOR 混淆，使攻擊者無法直接讀取或預測可用物件的地址。Linux 核心利用以下公式進行指標混淆：

```
ptr = ptr ^ kmem_cache->random ^ swab(ptr_addr);
```

Linux 核心使用 XOR 運算將原始指標 (指向下一個空間物件的地址) 與兩個值結合：一是每個 `kmem_cache` 初始化時設定的隨機數，二是儲存該指標的記憶體地址 (`ptr_addr`) 經過位元組順序交換 (`swab`，實作可參考 [uapi/linux/swab.h](#) 和 [linux/swab.h](#)) 後的結果。若存在一個漏洞允許攻擊者覆蓋 `ptr`，為了成功攻擊，需要知道隨機數和 `ptr_addr`，增加攻擊者的難度。

在本作業中，學生可以透過實作指標混淆得到加分機會。當同學在 [param.h](#) 中定義 `MP2_FREELIST_HARDENED = 1` 時：

- `kmem_cache` 內部將包含一個隨機數 `random`，用於 XOR 運算。
- `freelist` 指標在存取時需對指標進行解碼與編碼。

(2) freelist 隨機化 (Freelist Randomization)

在 SLAB 初始化時，隨機排列 `freelist` 內的可用物件，打破記憶體地址的順序性，提高安全性。

當同學在 [param.h](#) 中定義 `MP2_FREELIST_RANDOMIZATION = 1` 時：

- `freelist` 的初始化順序將由偽隨機數生成器生成。
- 偽隨機數生成器請自行實作，推薦可以實作於 [kernel/random.h](#) 中，作為 `xv6` 核心功能的擴展，未來將可能提供給其他核心開發者使用。
- 每個 `slab` 的 `freelist` 初始化順序將不同，增加攻擊難度。
- 我們會檢查初始化起始值的分布。

kmem_cache 的內部碎裂問題 (加分項目)

`struct kmem_cache` 本身屬於動態配置的系統物件，通常佔用一個完整的頁面，但其自身大小遠小於頁面大小，導致記憶體浪費。為解決此問題，可以將剩餘空間用來存放 Slab 內的可配置物件。

為了符合實作規範，請

- 將 `struct kmem_cache` 視為一個 `<slab_type>` 為 `cache` 的 `slab`。
- 列印的資訊中 `<slab_addr>` 直接對應 `struct kmem_cache` 自身在記憶體中的地址。
- 列印的資訊中 `<nxt_slab_addr>` 設為 `0x00...00`。

實作前需詳閱的資訊

檔案修改規則

- 請確保在 `student_id.txt` 檔案中填入您的學號。
- 以下分支中的受限制檔案 禁止修改：
 - 受限制的 Git 分支：`ntuos/mp2-submit`
 - 受限制的檔案：
 - `kernel/file.h`
 - `kernel/list.h`

- `user/` 目錄內的所有程式碼
- 任何對 `ntuos/mp2-submit` 分支內受限制檔案的變更將視為違規行為，並導致該次作業評分為零。
- 您可在其他 Git 分支進行修改。
- 允許在本機端對受限制檔案進行變更，但不得提交至受限制分支。
- `kernel/param.h` 檔案修改規範：
 - 學生 僅可變更 `MP2_FREELIST_HARDENED` 和 `MP2_FREELIST_RANDOMIZATION` 兩項設定。
 - 其他內容不得修改。
- `kernel/file.c` 檔案修改規範：
 - 禁止修改 任何 以 **[FILE]** 為前綴的除錯輸出程式碼。
 - 其他部分可進行調整，以使 `xv6` 使用 `struct kmem_cache` 管理 `struct file`。
- 除上述限制外，學生可自由新增檔案或修改其他程式碼。

簡易核心除錯器

在大型軟體專案中，為新增功能除錯是一項極具挑戰性的工作。為此，許多專案內建了日誌記錄或除錯系統，以協助開發人員進行問題排查。然而，`xv6` 僅提供基本的輸出功能，如 `printf`，這對於除錯與測試而言較為受限。因此，我們提供了一個簡易核心除錯系統，以提升除錯效率與測試靈活性，請同學們善加利用。

本除錯系統的使用方式與 `printf` 介面完全相容，可用於格式化輸出，且提供額外的控制選項。

```
// in file.c
#include "debug.h" // 引入核心除錯工具
// 格式化輸出
debug("tp: %d, ref: %d, readable: %d, ...",
      file->type, file->ref, file->readable, ...);
// 普通字串輸出
debug("[FILE] filealloc");
```

在本次 MP2 作業中，所有同學們的輸出 必須透過 `debug` 巨集函式，以確保測試與評分系統的正确運行。請務必嚴格遵守此要求，否則可能導致測試結果無法對應，影響成績。

詳細使用方式請參閱 [kernel/debug.h](#) 核心文件。

動態切換除錯模式

我們在 `xv6` 中提供了 `debugswitch` 命令，允許開發人員 透過命令行切換除錯模式。執行 `debugswitch` 後，系統將在 開啟/關閉除錯輸出 之間切換，便於測試與診斷。

以下為 `debugswitch` 的範例執行結果：

```
xv6 kernel is booting

[FILE] fileinit
hart 2 starting
```

```

hart 1 starting
[FILE] filealloc
init: starting sh
[FILE] filealloc
[FILE] fileclose
$ debugswitch
Switch debug mode to 0
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2292
cat        2 3 34728
...
console    3 22 0
$ debugswitch
Switch debug mode to 1
$ ls
[FILE] filealloc
[FILE] filealloc
[FILE] fileclose
.          1 1 1024
[FILE] filealloc
[FILE] fileclose
..         1 1 1024
[FILE] filealloc
[FILE] fileclose
README    2 2 2292
...
[FILE] filealloc
[FILE] fileclose
console    3 22 0
[FILE] fileclose

```

調整預設除錯模式

預設的除錯模式可透過 `param.h` 進行設定。

開發時，同學們可根據需求調整本機的除錯模式，但請確保提交至 **GitHub** 的 `MP2_DEFAULT_DEBUG_MODE` 為 `1`，即開啟除錯模式，以確保評分系統的正确性。

```

// in param.h
#define MP2_DEFAULT_DEBUG_MODE 1 // 預設開啟除錯模式

```

實作要求

本作業提供學生們高度的靈活性，允許對 `slab` 進行自訂設計，只需遵循以下規範。

struct slab 設計

請設計 struct slab 資料結構。由於核心開發者通常傾向於最大化記憶體利用率，因此建議學生最小化 struct slab 的記憶體佔用，以提升對記憶體空間的利用率。

```
struct slab {
    <ptr> freelist;

    {
        <ptr> <next>;
        <ptr> <prev>;
    }
    ... // 允許學生自由擴展
};
```

struct slab 的設計將依據以下三個標準進行評分：

1. struct slab 本身的記憶體大小 (最高 9%)

定義 struct slab 的大小因子 $v(s)$ 如下：

$$v(s) = \frac{\text{sizeof}(\text{struct slab})}{\text{sizeof}(\text{void}^*)}$$

評分標準如下：

$v(s)$	分數
≤ 3	3% + 6%
4	3% + 2%
5	3%
6	2%
7	1%
≥ 8	0%

2. slab::freelist 中可容納的物件數量 (2%)

在測試過程中，struct file 的大小為 504 Bytes，評分標準如下：

可容納 struct file 數量	分數
8	2%
7	1%
≤ 6	0%

3. 使用 struct list_head 進行 Slab 管理 (加 10%)

- 需在 struct slab 中使用 **struct list_head** 維護 Slab 間的鏈結。

- 同學的程式碼 需在功能測試的部分取得滿分 (45%) 才會獲得這部分額外 10% 的加分

struct kmem_cache 的設計

```
struct kmem_cache {
    char name[MP2_CACHE_MAX_NAME];
    uint object_size;
    struct spinlock lock;

    <ptr> full; // 全滿 (可選)
    <ptr> partial; // 部分使用中
    <ptr> free; // 閒置 (可選)

    ... // 允許學生自由擴展
};
```

請同學留意以下幾點：

1. 閒置 Slab 釋放機制

為了減少過多閒置 Slab 的記憶體浪費，當可用 Slab（即 `partial + free`）的總數超過 `param.h` 中定義的 `MP2_MIN_AVAIL_SLAB` 時，若有新的 Slab 變為完全閒置（`free`），則應主動釋放其佔用的記憶體。該機制將透過 `kmem_cache_free` 進行測試。

2. 內部碎裂優化

由於內部碎裂問題，若透過 `kmem_cache` 的內部空間進行物件的配置與釋放（即將這些物件的 `<slab_addr>` 設為 `kmem_cache` 的地址），可獲得額外 7% 的加分。

3. `full` 和 `free` 的可選性

`kmem_cache` 的 `full` 與 `free` 變數為可選項，學生可以參考 [Linux Kernel 中 SLUB 的設計方式](#)，或採用其他適合的實作方法，只要符合實作規範，即可獲得相應評分。

Slab 提供的功能

請在 `slab.c` 中實作以下函式：

```
// 1. Slab 記憶體管理核心函式
// 1-1. 初始化 Slab 配置器，為類型名稱為 name 的系統物件（大小為 object_size）創建一個
kmem_cache
struct kmem_cache *kmem_cache_create(char *name, uint object_size);
// 1-2. 配置一個系統物件並返回其記憶體地址
void *kmem_cache_alloc(struct kmem_cache *cache);
// 1-3. 釋放指定的系統物件 (obj)
void kmem_cache_free(struct kmem_cache *cache, void *obj);
// 1-4. 銷毀 kmem_cache
void kmem_cache_destroy(struct kmem_cache *cache);

// 2. 除錯用函式
void print_kmem_cache(struct kmem_cache *, void (*)(void *));
```

上述所有 Slab 記憶體管理功能，在輸出訊息時，皆應使用 [SLAB] 作為前綴。例如：

```
[SLAB] Alloc request on cache file
```

kmem_cache_create: 創建 kmem_cache

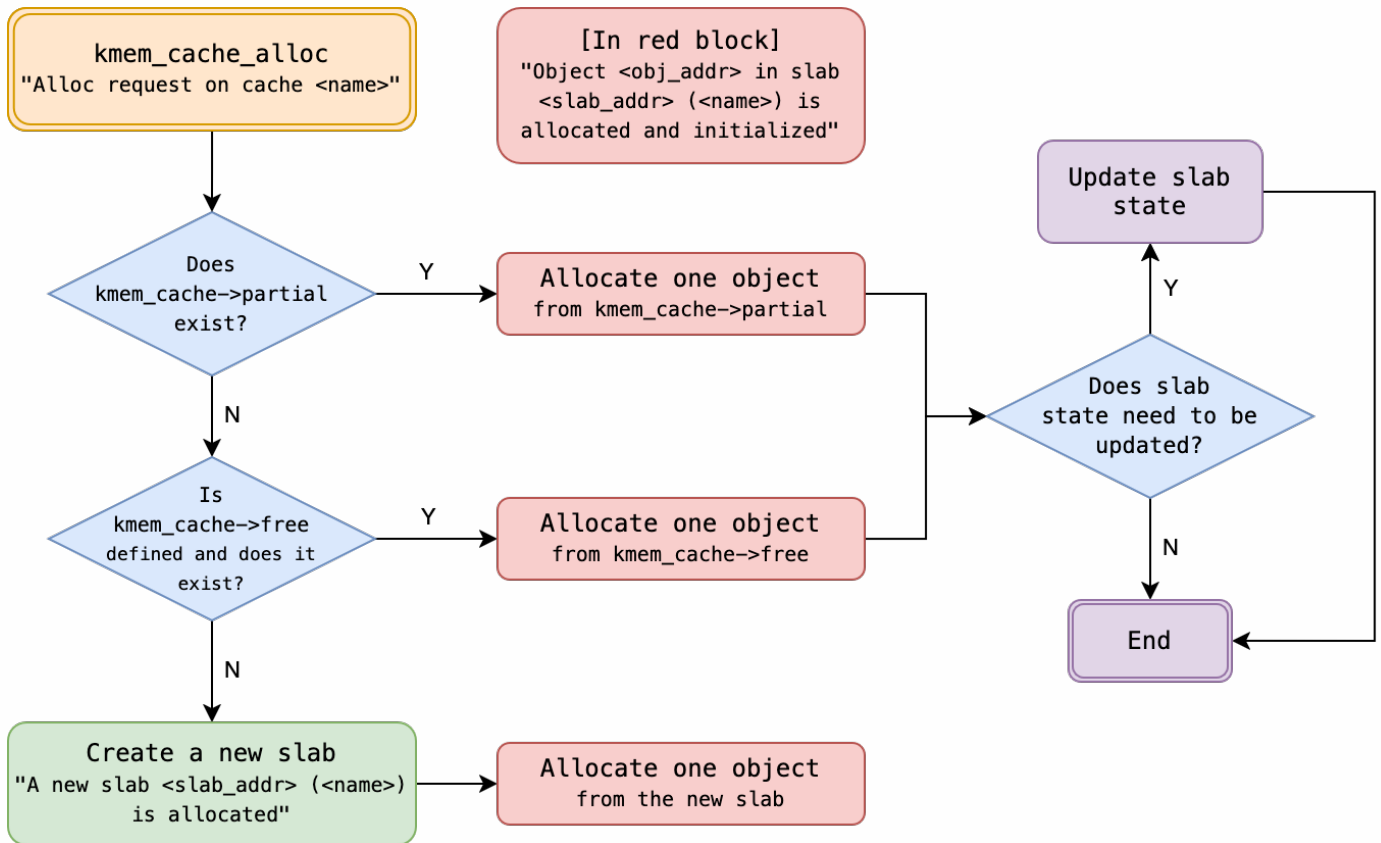
在成功創建並返回 kmem_cache 之前，請輸出以下資訊：

```
[SLAB] New kmem_cache (name: <name>, object size: <obj_size> bytes, max objects per
slab: <max_objs>) is created
```

- **<name>**：新建的 kmem_cache 之名稱 (kmem_cache::name)。
- **<obj_size>**：該 kmem_cache 內部物件的大小 (kmem_cache::object_size，單位為 Bytes)。
- **<max_objs>**：一個 slab 中能容納物件的最大數量。

kmem_cache_alloc: 配置物件

在配置物件時，請依照以下流程圖執行，並輸出相應資訊。請將尖括號 (<>) 內的變數替換為實際數值，並確保每行輸出皆以 [SLAB] 為前綴，每個字中間間隔一個空白。

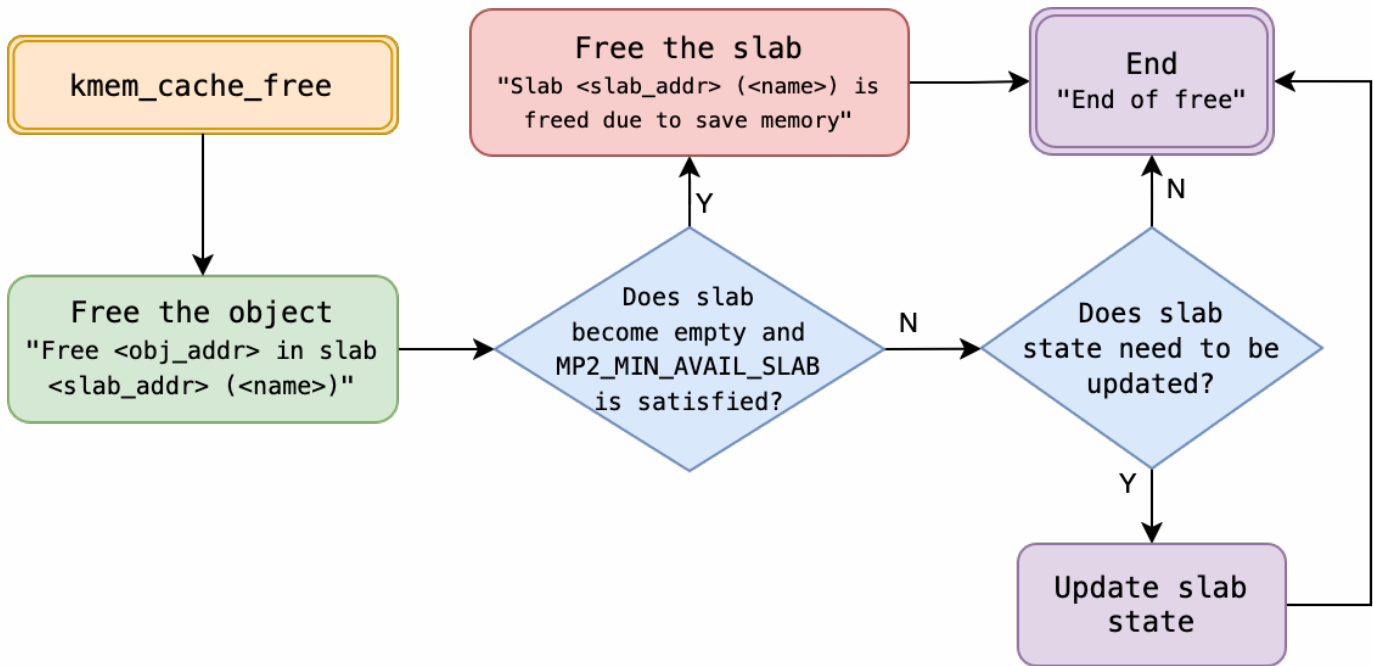


- **<name>** : kmem_cache 的名稱 (kmem_cache::name)。
- **<slab_addr>** : 該物件所屬 Slab 的記憶體地址。
- **<obj_addr>** : 配置的物件所在記憶體地址。

除此之外，同學們也可以列印其他自訂的除錯訊息，只要不和流程圖中出現的列印格式相衝突即可。建議以其他前綴 (比如小寫的 [slab] 等) 列印自定義的除錯訊息。

kmem_cache_free: 釋放物件

在釋放物件時，請依照以下流程圖執行，並輸出相應資訊。請確保輸出符合規範，並替換 尖括號 (<>) 內的變數，所有訊息均需加上 [SLAB] 前綴，每個字中間間隔一個空白。



- **<name>** : kmem_cache 的名稱 (kmem_cache::name)。
- **<slab_addr>** : 該物件所屬 Slab 的記憶體地址。
- **<obj_addr>** : 待釋放的物件所在記憶體地址。
- **<before>** : 物件所屬 Slab 在釋放前的狀態 (full/partial/free/cache)。
- **<after>** : 物件所屬 Slab 在釋放後的狀態 (full/partial/free/cache)。

此外，若 (**partial + free Slab 數量**) 超過 **MP2_MIN_AVAIL_SLAB**，且該物件所在 Slab 變為完全閒置 (free)，則應釋放該 Slab 以回收記憶體。

除此之外，同學們也可以列印其他自訂的除錯訊息，只要不和流程圖中出現的列印格式相衝突即可。建議以其他前綴 (比如小寫的 [slab] 等) 列印自定義的除錯訊息。

將 slab 配置器應用於 struct file 的管理

在 xv6 中，原本負責管理 struct file 的結構為 file.c 中的 ftable。請將其替換為 struct kmem_cache *file_cache，並相應調整 file.c 的實現。

在 file.[h,c] 中，已為同學添加以下程式碼：

```

// file.h
#include "kernel/param.h"
#include "kernel/slab.h"

struct file {
    ...
#ifdef MP2_TEST
    int fat_element[MP2_FILE_MAGIC_N];
#endif // _MP2_TEST_
};

extern struct kmem_cache *file_cache;
// 列印檔案物件的元資料

```

```

void fileprint_metadata(void *f);

// file.c
#include "kernel/debug.h"
void fileprint_metadata(void *f) {
    struct file *file = (struct file *) f;
    debug("tp: %d, ref: %d, readable: %d, writable: %d, pipe: %p, ip: %p, off: %d, major: %d",
        file->type, file->ref, file->readable, file->writable, file->pipe, file->ip,
        file->off, file->major);
}

struct kmem_cache *file_cache;

void
fileinit(void)
{
    debug("[FILE] fileinit\n");
    // ...
}

// Allocate a file structure.
struct file*
filealloc(void)
{
    debug("[FILE] filealloc\n");
    // ...
}

// ...

void
fileclose(struct file *f)
{
    // ...
    debug("[FILE] fileclose\n");
    ff = *f;
    f->ref = 0;
    // ...
}

```

請同學們 不要更動 在 `file.c` 中添加的 列印相關程式碼，在 `fileinit`、`filealloc` 和 `fileclose` (ref 降至 0 時) 應該印出以 `[FILE]` 為前綴的除錯訊息。

print_kmem_cache 列印 *struct kmem_cache* 的資訊

在列印 `struct kmem_cache` 時，應根據 `struct slab` 配置剩餘數量的狀態，將 `struct slab` 分類為 `<slab_type>`（例如 `full`、`partial` 等），並採用以下格式：

```
[SLAB] kmem_cache { name: <name>, obj_size: <object_size>, harden: <harden>, rand:
<rand> }
[SLAB] <SPACE>[ <slab_type> slabs ]
...
[SLAB] <SPACE>[ <slab_type> slabs ]
...
```

同時，需列印系統生成並管理的所有 struct slab，每個 struct slab 應包含以下資訊：

```
[SLAB] <SPACE>[ slab <slab_addr> ] { freelist: <freelist>, nxt: <next_slab_addr> }
[SLAB] <SPACE>[ idx <idx> ] { addr: <entry_addr>, as_ptr: <as_ptr>, as_obj: {<as_obj>} }
[SLAB] <SPACE>[ idx <idx> ] { addr: <entry_addr>, as_ptr: <as_ptr>, as_obj: {<as_obj>} }
...
[SLAB] <SPACE>[ idx <idx> ] { addr: <entry_addr>, as_ptr: <as_ptr>, as_obj: {<as_obj>} }
```

其中各欄位定義如下：

- <name>：kmem_cache 的名稱 (kmem_cache::name)。
- <obj_size>：kmem_cache 中物件的大小 (kmem_cache::object_size)。
- <SPACE>：一個以上任意數量的空格 () 或 \t。
- <slab_type>：分類為 full、partial、free 或 **cache**。
 - 除錯時可列印所有類型。
 - 實際測試僅檢查 **partial** 和 **cache** (若有實現) 的部分。
 - full 和 free slab 可以透過推論來追蹤，故不必印出
- <slab_addr>：slab 在記憶體中的地址。
- <nxt_slab_addr>：對應的 slab 的下一個 slab 在記憶體中的地址。
- <harden>：MP2_FREELIST_HARDENED 的值 (0 或 1)，請參考 [freelist 安全性議題](#)。
- <rand>：MP2_FREELIST_RANDOMIZATION 的值 (0 或 1)，請參考 [freelist 安全性議題](#)。
- <freelist>：slab::freelist 的值。
- <entry_addr>：根據 slab::freelist 中各物件的記憶體地址，按由小到大的順序列印每個物件的地址，相當於其在 freelist 中的 entry。
- <as_ptr>：將該 entry 的物件解讀為指標的結果。
- <as_obj>：將該 entry 的物件解讀為系統物件的結果，即將物件傳入 slab_obj_printer 的輸出。

舉例如下。

```
[SLAB] kmem_cache { name: file, object_size: 504, harden: 0, rand: 1 }
[SLAB] [ full slabs ]
[SLAB] [ partial slabs ]
[SLAB] [ slab 0x0000000087f4e000 ] { freelist: 0x0000000087f4ede8, prev: 0x0000000087f59040, nxt: 0x0000000087f32008 }
[SLAB] [ idx 0 ] { addr: 0x0000000087f4e028, as_ptr: 0x0000000000000000, as_obj: { tp: 3, ref: 9, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x00000000000035408, off: 0, major: 1 } }
[SLAB] [ idx 1 ] { addr: 0x0000000087f4e218, as_ptr: 0x0000000000000000, as_obj: { tp: 0, ref: 0, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x00000000000035508, off: 0, major: 0 } }
[SLAB] [ idx 2 ] { addr: 0x0000000087f4e410, as_ptr: 0x0000000087f4e218, as_obj: { tp: -2013994472, ref: 0, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x00000000000035620, off: 0, major: 0 } }
[SLAB] [ idx 3 ] { addr: 0x0000000087f4e600, as_ptr: 0x0000000087f4e410, as_obj: { tp: -2013993968, ref: 0, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x000000000000356a8, off: 0, major: 0 } }
[SLAB] [ idx 4 ] { addr: 0x0000000087f4e800, as_ptr: 0x0000000087f4e600, as_obj: { tp: -2013993464, ref: 0, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x00000000000035730, off: 0, major: 0 } }
[SLAB] [ idx 5 ] { addr: 0x0000000087f4e9f8, as_ptr: 0x0000000087f4e800, as_obj: { tp: -2013992960, ref: 0, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x000000000000357b8, off: 0, major: 0 } }
[SLAB] [ idx 6 ] { addr: 0x0000000087f4ebf0, as_ptr: 0x0000000087f4e9f8, as_obj: { tp: -2013992456, ref: 0, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x00000000000035840, off: 0, major: 0 } }
[SLAB] [ idx 7 ] { addr: 0x0000000087f4ede8, as_ptr: 0x0000000087f4ebf0, as_obj: { tp: -2013991952, ref: 0, readable: 1, writable: 1, pipe: 0x0000000000000000, ip: 0x000000000000358c8, off: 0, major: 0 } }
[SLAB] [ free slabs ]
```

學生可在列印的資訊中自由添加除要求外的信息，只要包含必列的資訊即可，且順序可自行調整。

```
[SLAB] kmem_cache { <KV_PAIRS> }
[SLAB] <SPACE>[ slab <slab_addr> ] { <KV_PAIRS> }
[SLAB] <SPACE>[ idx <idx> ] { <KV_PAIRS> }
[SLAB] <SPACE>[ idx <idx> ] { <KV_PAIRS> }
...
[SLAB] <SPACE>[ idx <idx> ] { <KV_PAIRS> }
```

<KV_PAIRS> 表示不定數量的 key: value 鍵值對，以 <SPACE>, <SPACE> 分隔。

舉例如下。

```
[SLAB] kmem_cache { rand: 0, name: file, some_thing: meow, object_size: 504, another_thing: 123, harden: 1 }
[SLAB] [ cache slabs ]
[SLAB] [ slab 0x0000000087f59000 ] { freelist: 0x0000000087f59268, nxt: 0x0000000000000000 }
[SLAB] [ idx 0 ] { addr: 0x0000000087f59070, as_ptr: 0x0000000090000003, as_obj: { tp: 3, ref: 9, readable: 1, writable: 1, pipe: 0x0505050505050505, ip: 0x0000000000035558, off: 84215045, major: 1 } }
[SLAB] [ idx 1 ] { addr: 0x0000000087f59268, as_ptr: 0x0000000087f59460, as_obj: { tp: -201394832, ref: 0, readable: 1, writable: 1, pipe: 0x0505050505050505, ip: 0x0000000000035558, off: 84215045, major: 1 } }
[SLAB] [ idx 2 ] { addr: 0x0000000087f59460, as_ptr: 0x0000000087f59658, as_obj: { tp: -201394832, ref: 0, readable: 5, writable: 5, pipe: 0x0505050505050505, ip: 0x0505050505050505, off: 84215045, major: 1285 } }
[SLAB] [ idx 3 ] { addr: 0x0000000087f59658, as_ptr: 0x0000000087f59850, as_obj: { tp: -2013947824, ref: 0, readable: 5, writable: 5, pipe: 0x0505050505050505, ip: 0x0505050505050505, off: 84215045, major: 1285 } }
[SLAB] [ idx 4 ] { addr: 0x0000000087f59850, as_ptr: 0x0000000087f59a48, as_obj: { tp: -2013947328, ref: 0, readable: 5, writable: 5, pipe: 0x0505050505050505, ip: 0x0505050505050505, off: 84215045, major: 1285 } }
[SLAB] [ idx 5 ] { addr: 0x0000000087f59a48, as_ptr: 0x0000000087f59c40, as_obj: { tp: -2013946816, ref: 0, readable: 5, writable: 5, pipe: 0x0505050505050505, ip: 0x0505050505050505, off: 84215045, major: 1285 } }
[SLAB] [ idx 6 ] { addr: 0x0000000087f59c40, as_ptr: 0x0000000000000000, as_obj: { tp: 0, ref: 0, readable: 5, writable: 5, pipe: 0x0505050505050505, ip: 0x0505050505050505, off: 84215045, major: 1285 } }
[SLAB] [ idx 7 ] { addr: 0x0000000087f59e38, as_ptr: 0x0505050505050505, as_obj: { tp: 84215045, ref: 84215045, readable: 5, writable: 5, pipe: 0x0505050505050505, ip: 0x0505050505050505, off: 84215045, major: 1285 } }
[SLAB] [ full slabs ]
[SLAB] [ partial slabs ]
[SLAB] [ free slabs ]
```

實作 *system call sys_printfslab*

請同學們實現 `sys_printfslab`，印出 `file` 之 `slab` (`struct kmem_cache`) 的系統呼叫。

請確保下列 `user program` 能正常運行。

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
    printfslab();
}
```

上述程式碼只要能被正常編譯，無論是否印出正確的 `slab`，都能得到這部分的分數。預設還無法編譯成功。

參考資料

1. [xv6: a simple, Unix-like teaching operating system](#)
2. [ISO/IEC 9899:2024 \(C 語言規格書\)](#)
3. [linux/mm/slab.h](#)
4. [linux/mm/slub.c](#)
5. [sysprog21/lab0-c](#)
6. [linux/include/linux/list.h](#)
7. [slab 記憶體配置器](#)